

SB STOCKS

Full-Stack Stock Trading Simulator

Technical Documentation

MERN Stack | React 18 + Vite | Node.js + Express | MongoDB

1. Project Description

SB Stocks is a full-stack stock trading simulator that lets users trade with virtual funds, track a live portfolio, build watchlists with price alerts, set financial goals, and compete on a performance leaderboard. The frontend is a React (Vite) single-page app with a 3D animated landing page and smooth page transitions; the backend is a Node/Express REST API backed by MongoDB.

All trading uses virtual funds (walletBalance, seeded at ₹1,00,000 per user). Stock prices are not sourced from a live market feed — they are generated by a lightweight price-simulation service that runs a random-walk tick every 15 seconds, updating prices, day-high/low, 52-week range, and market-data history in real time.

1.1 Technology Stack

Layer	Technologies
Frontend	React 18, Vite, Tailwind CSS, Redux Toolkit, React Router v6, Framer Motion, React Three Fiber + Drei (3D), Recharts, Axios, jsPDF + jspdf-autotable
Backend	Node.js, Express 4, MongoDB + Mongoose 8, JWT (jsonwebtoken), bcryptjs, express-validator, Morgan, CORS, dotenv
Dev Tooling	Nodemon (server hot-reload), Vite dev server (client), ESM (client) / CommonJS (server)

1.2 Application Pages

- Public: Landing, Login, Register, Stocks, Stock Details, 404
- Authenticated (user): Dashboard, Portfolio, Watchlist, Compare, Goals, Leaderboard, Transactions, Profile, Settings
- Authenticated (admin): Admin Dashboard, Admin Users, Admin Stocks, Admin Logs

2. Modules & Features

The platform is organized into functional modules spanning authentication, trading, portfolio tracking, watchlists, goals, leaderboard, notifications, and administration. The diagram below (from the project's architecture assets) summarizes the feature set.

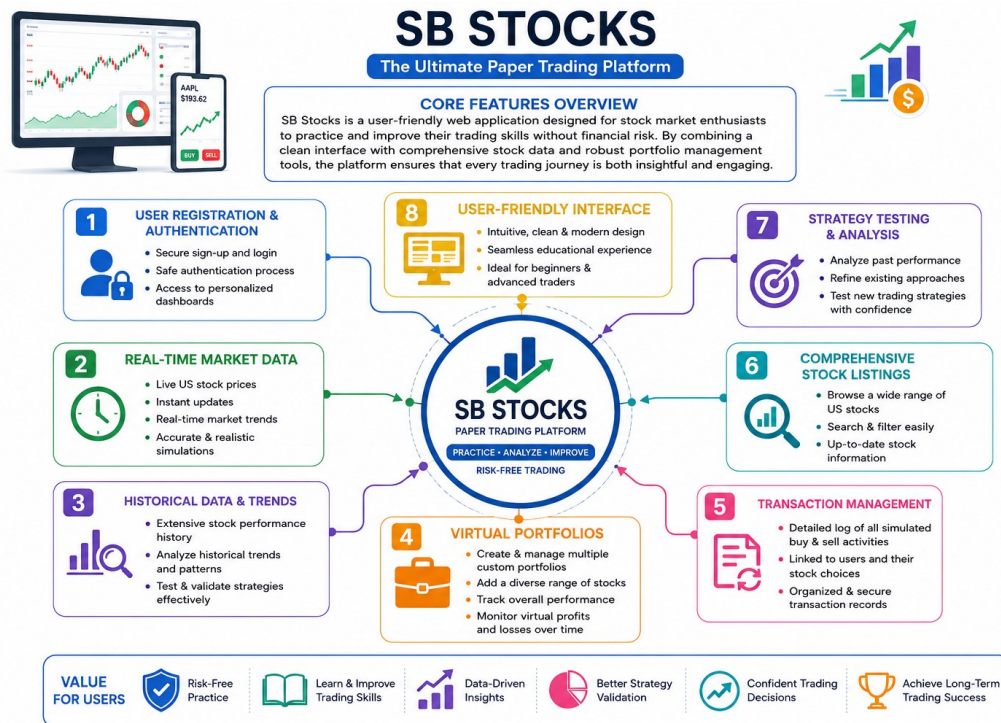


Figure 2.1 — SB Stocks Feature Map

2.1 Module Summary

Module	Responsibility
Auth	Registration, login, JWT issuance, session verification (/api/auth)
Trading	Buy / sell execution against live stock price and wallet balance (/api/trading)
Stocks	Stock catalog, search/filter/sort, price history (/api/stocks)
Portfolio	Live holdings valuation, P/L, allocation, net worth (/api/portfolio)
Watchlist	Track stocks, pin favorites, set price alerts (/api/watchlist)
Goals	User-defined financial targets with live progress tracking (/api/goals)
Transactions	Filterable buy/sell history per user (/api/transactions)
Leaderboard	Ranks users by live ROI computed from holdings + wallet (/api/leaderboard)
Notifications	In-app price-alert notifications (/api/notifications)
Admin	User/stock management, platform analytics, activity logs (/api/admin)

3. Folder Structure

SB Stocks follows a client/server monorepo layout — a React client and an Express server as independent npm packages under one root.

```

SB-Stocks/
├── client/                                # React frontend (Vite)
│   ├── src/
│   │   ├── assets/                      # Static assets
│   │   ├── components/                  # Reusable UI components (20 files)
│   │   ├── pages/                       # Route-level views (19 pages)
│   │   └── redux/slices/                 # Redux Toolkit slices (6 slices)

```

```

|— services/      # Axios API client (api.js)
|— hooks/        # Custom React hooks
|— layouts/      # MainLayout, AuthLayout, AdminLayout
|— routes/       # ProtectedRoute, AdminRoute guards
|— utils/        # Formatters, PDF export, chart theme, etc.
|— server/       # Express backend
|   |— config/   # db.js (Mongoose connection)
|   |— controllers/ # 10 route controllers
|   |— middleware/ # auth, admin, errorHandler
|   |— models/   # 11 Mongoose schemas
|   |— routes/   # 11 route modules
|   |— services/ # stockPriceService, alertService
|   |— utils/    # asyncHandler, generateToken, formatCurrency
|   |— validators/ # request validation (express-validator)
|   |— seed.js   # sample stock seeder
|   |— app.js    # Express app + route mounting
|   |— server.js # entry point, DB connect, price simulation
|— README.md
|— package.json  # root scripts (install-all, server, client)

```

4. MVC Architecture

The backend follows an MVC-inspired layered pattern: Mongoose Models define data, Controllers hold business logic, and Express Routes map HTTP endpoints to controllers. The React client renders Views and dispatches actions to Redux, which calls the API layer — mirroring the same separation on the frontend.

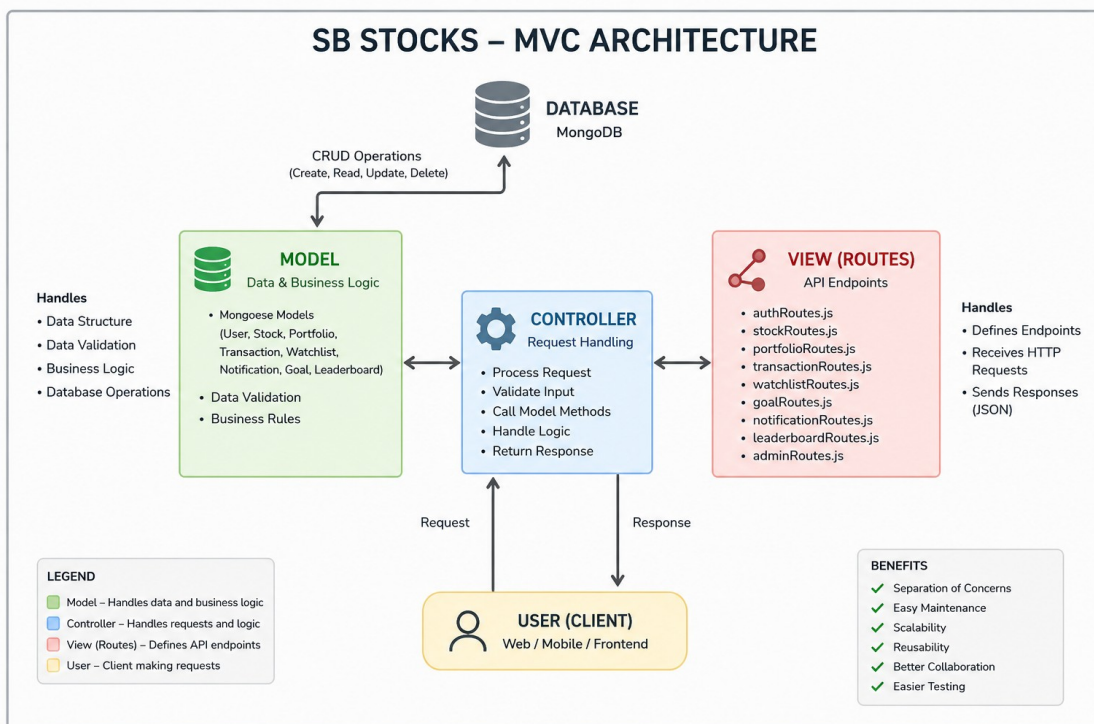


Figure 4.1 — SB Stocks MVC Pattern

5. Technical Architecture

SB Stocks is a three-tier MERN application: a React SPA client, an Express REST API, and MongoDB persistence, with an internal price-simulation service standing in for a live market-data feed.

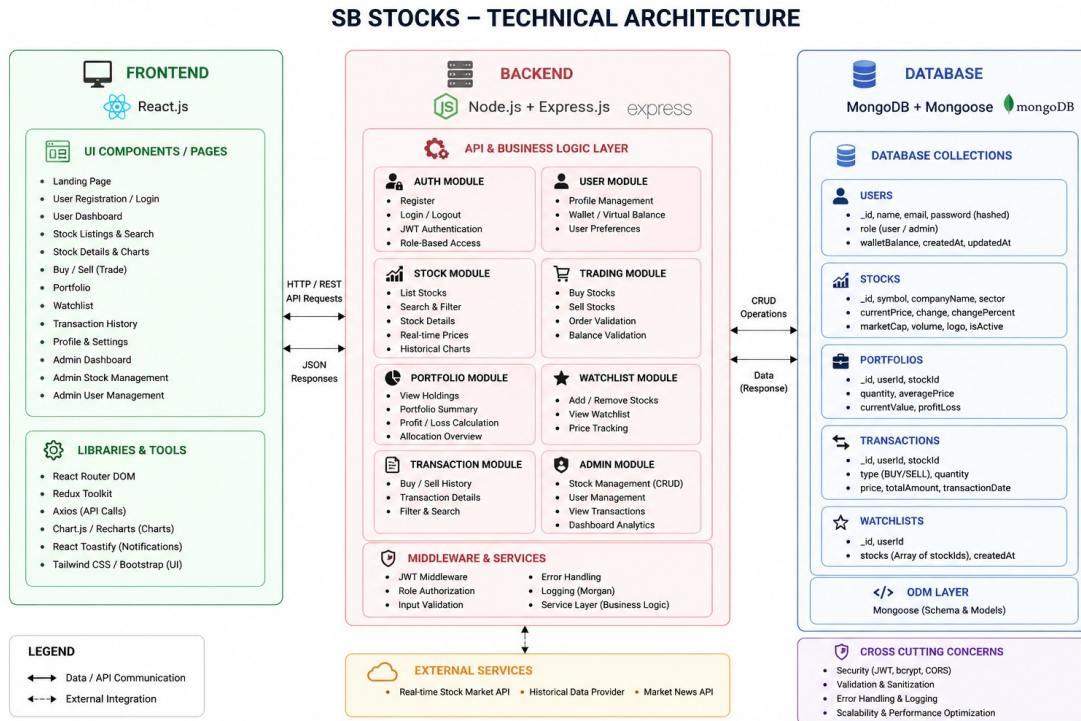


Figure 5.1 — SB Stocks Technical Architecture

5.1 Key Backend Building Blocks

Component	File	Purpose
App bootstrap	app.js	Configures CORS, JSON body parsing, Morgan logging, mounts all /api routes, wires error middleware
Entry point	server.js	Loads env vars, connects MongoDB, clears stale notifications, starts the HTTP server and price simulation loop
DB connection	config/db.js	Establishes the Mongoose connection to MONGO_URI
Price engine	services/stockPriceService.js	Random-walk price ticker ($\pm 1.2\%$) every 15s; updates Stock + writes MarketData; triggers alert checks
Alert engine	services/alertService.js	Evaluates active watchlist price alerts against each tick and creates Notifications
Auth middleware	middleware/auth.js	Verifies JWT, attaches req.user
Admin middleware	middleware/admin.js	Restricts routes to role === "admin"
Error handling	middleware/errorHandler.js	notFound (404) + centralized errorHandler (JSON error responses)
Async wrapper	utils/asyncHandler.js	Forwards async controller errors to errorHandler without try/catch boilerplate

6. User Flow

New visitors land on the 3D animated landing page and can browse public stock listings before authenticating. Registration auto-provisions a Portfolio and Watchlist; once logged in, users trade, monitor holdings, set goals, and receive price-alert notifications.

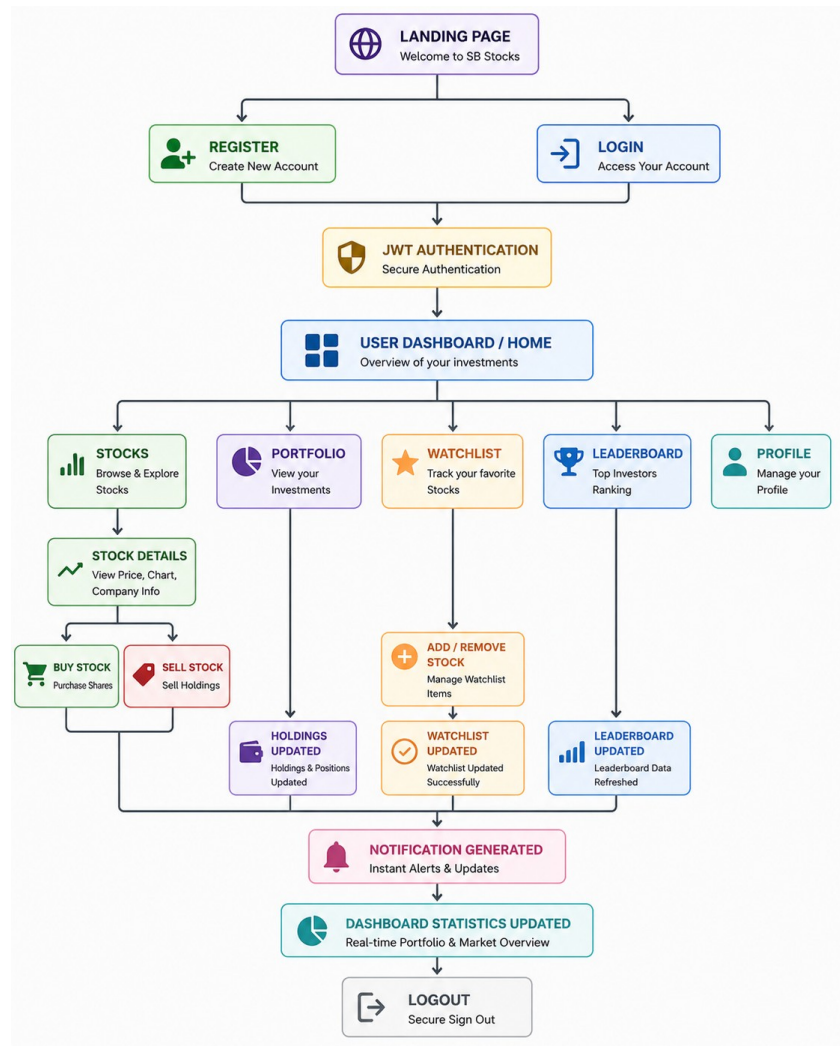


Figure 6.1 — SB Stocks User Flow

7. Database Design (ER Diagram)

SB Stocks persists 11 Mongoose collections. Relationships are maintained via ObjectId references (ref) rather than embedded documents, keeping holdings, transactions, and market data independently queryable.

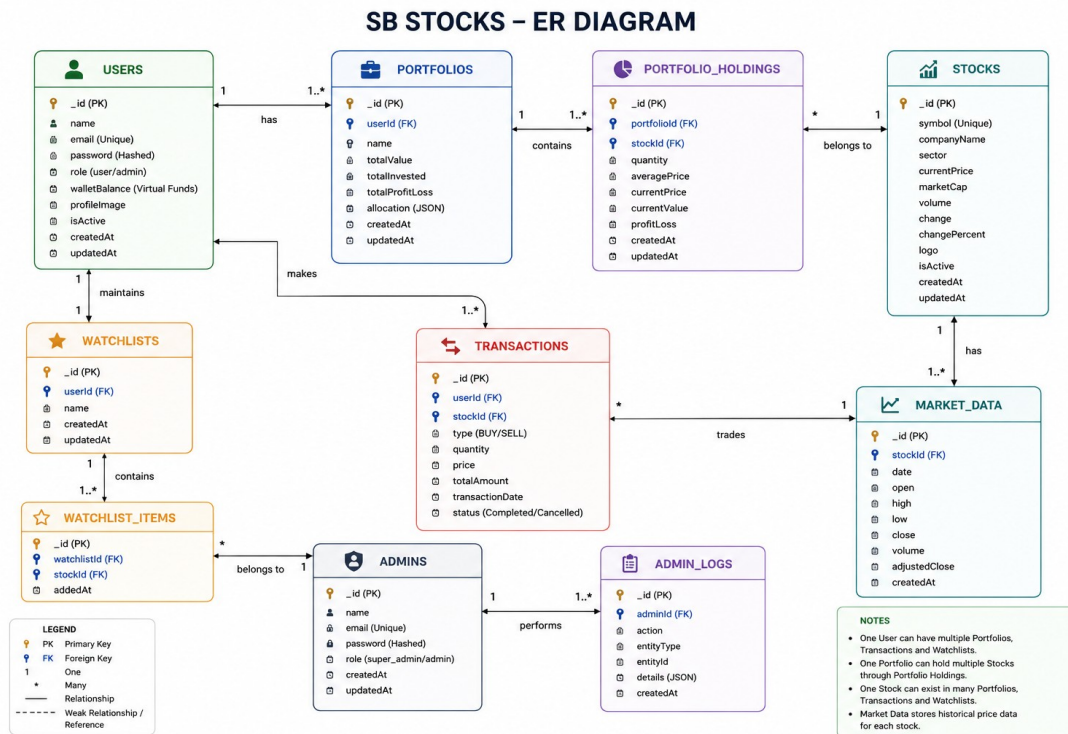


Figure 7.1 — SB Stocks Entity-Relationship Diagram

7.1 Collection Reference

Model	Key Fields	Relationships
User	name, email (unique), password (hashed, select:false), role [user admin], walletBalance (default 100000), profileImage, isActive	1:1 → Portfolio, 1:1 → Watchlist
Stock	symbol (unique), companyName, sector, currentPrice, change, changePercent, marketCap, volume, isFeatured, isTrending, price-stat fields (openPrice, dayHigh/Low, week52High/Low, peRatio, eps, beta), company-info fields (about, ceo, founded, etc.)	Referenced by PortfolioHolding, WatchlistItem, Transaction, MarketData
Portfolio	userId (unique), totalValue, totalInvested, totalProfitLoss, allocation, totalInvestment, todayInvestment, todayInvestmentDate	1:1 → User; 1:N → PortfolioHolding
PortfolioHolding	portfolioId, stockId, quantity, averagePrice, currentValue, profitLoss	Unique index (portfolioId, stockId)
Watchlist	userId (unique), name	1:1 → User; 1:N → WatchlistItem
WatchlistItem	watchlistId, stockId, pinned, targetPrice, alertDirection [above below], alertTriggeredAt	Unique index (watchlistId, stockId)
Transaction	userId, stockId, type [BUY SELL], quantity, price, totalAmount, costBasis (SELL only), transactionDate, status	N:1 → User, N:1 → Stock
Goal	userId, title, type [SAVINGS PORTFOLIO_VALUE PROFIT MONTHLY_INVESTMENT CUSTOM], targetAmount, manualAmount, deadline, achieved, achievedAt	N:1 → User
Notification	userId, type [PRICE_ALERT], stockId, title, message, read	N:1 → User, N:1 → Stock; compound index (userId, createdAt)
MarketData	stockId, date, open, high, low, close, volume, adjustedClose	N:1 → Stock (OHLC history for charts)
AdminLog	adminId, action, entityType, entityId, details	N:1 → User (admin audit)

Model	Key Fields	Relationships
		trail)

8. Roles & Responsibilities

SB Stocks defines two roles: user and admin. There is no public admin sign-up — an account is promoted to admin by manually setting role: "admin" on the User document.

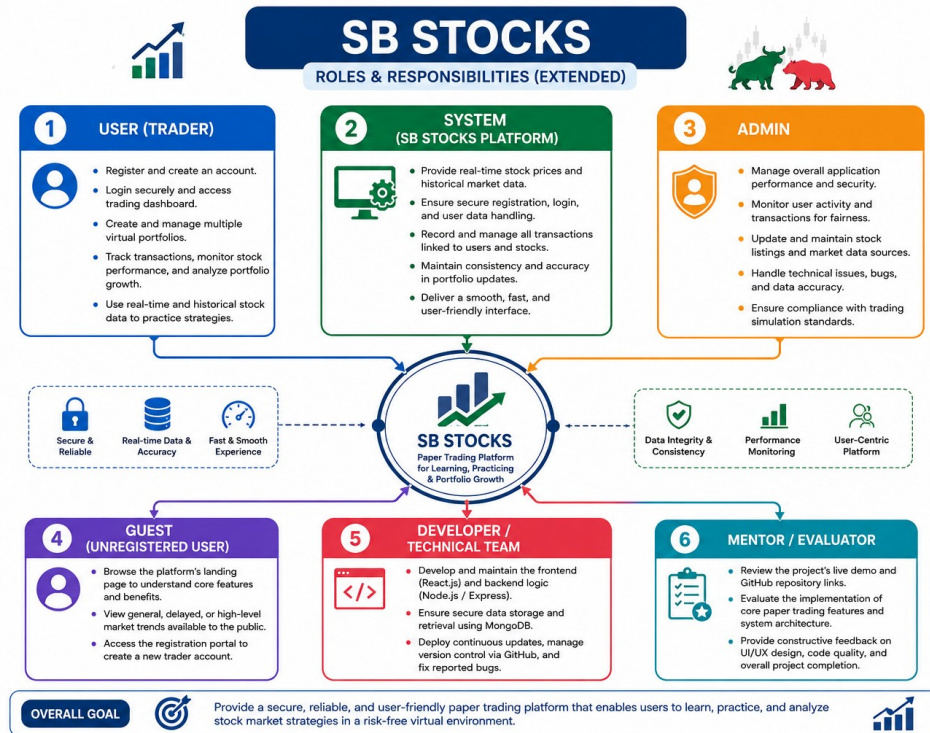


Figure 8.1 — SB Stocks Roles & Responsibilities

8.1 Access Control Summary

Role	Guarded By	Capabilities
Guest	— (no token)	View landing page, browse stock listings and stock details
User	protect middleware (valid JWT)	Trade, view/manage portfolio, watchlist, goals, transactions, leaderboard, profile & settings
Admin	protect + admin middleware (role === "admin")	Manage users & stocks, view platform-wide analytics, transactions and activity logs

9. API Reference

All endpoints are prefixed with /api. Protected routes require an Authorization: Bearer <token> header; admin routes additionally require an admin role.

9.1 Auth — /api/auth

Method	Endpoint	Access	Description
POST	/register	Public	Create account; auto-creates Portfolio +

Method	Endpoint	Access	Description
			Watchlist; returns JWT
POST	/login	Public	Authenticate; returns JWT + user profile
GET	/me	Private	Return the logged-in user

9.2 Users — /api/users

Method	Endpoint	Access	Description
PUT	/me	Private	Update name / profileImage
PUT	/me/password	Private	Change password (verifies current password)

9.3 Stocks — /api/stocks

Method	Endpoint	Access	Description
GET	/	Public	List/search/filter/sort stocks (search, sector, sort=gainers losers volume)
GET	/:id	Public	Stock detail + last 3000 MarketData points (chronological)

9.4 Trading — /api/trading

Method	Endpoint	Access	Description
POST	/buy	Private	Buy shares at current price; updates holding, wallet, portfolio investment totals
POST	/sell	Private	Sell shares; validates holding quantity; records cost basis on the transaction

9.5 Portfolio, Watchlist, Goals, Transactions

Method	Endpoint	Access	Description
GET	/api/portfolio	Private	Live holdings, valuation, today's/overall P&L, net worth
GET	/api/watchlist	Private	Get (or auto-create) watchlist + items
POST	/api/watchlist	Private	Add a stock to the watchlist
PATCH	/api/watchlist/:stockId	Private	Pin/unpin, set or clear a price alert
DELETE	/api/watchlist/:stockId	Private	Remove a stock from the watchlist
GET	/api/goals	Private	List goals with live-computed progress
POST	/api/goals	Private	Create a goal
PUT	/api/goals/:id	Private	Edit a goal
DELETE	/api/goals/:id	Private	Delete a goal

Method	Endpoint	Access	Description
GET	/api/transactions	Private	User's transactions (filter by type, from, to)

9.6 Leaderboard & Notifications

Method	Endpoint	Access	Description
GET	/api/leaderboard	Private	Users ranked by live ROI (net worth vs. ₹1,00,000 base)
GET	/api/notifications	Private	Latest 50 notifications + unread count
PATCH	/api/notifications/read-all	Private	Mark all notifications as read
DELETE	/api/notifications	Private	Clear all notifications

9.7 Admin — /api/admin (requires admin role)

Method	Endpoint	Description
GET	/dashboard	Platform stats, 14-day user growth & buy/sell volume, recent transactions
GET	/logs	Latest 200 admin activity logs
GET	/users	List all users
PUT	/users/:id	Update a user's role / active status
DELETE	/users/:id	Delete a user
POST	/stocks	Create a stock
PUT	/stocks/:id	Update a stock
DELETE	/stocks/:id	Delete a stock
GET	/transactions	View all transactions platform-wide

10. Authentication & Authorization

10.1 Mechanism

- Passwords are hashed with bcryptjs (10 salt rounds) in a User pre("save") hook; the password field is select: false by default.
- On register/login, generateToken() signs a JWT ({ id, role }) with JWT_SECRET, expiring per JWT_EXPIRES_IN (default 7d).
- The client stores the token in localStorage (sb_token) and Axios attaches Authorization: Bearer <token> on every request via a request interceptor.
- The protect middleware verifies the JWT, loads the user (minus password) and attaches it to req.user; missing/invalid/expired tokens return 401.
- The admin middleware runs after protect and returns 403 unless req.user.role === "admin".
- A response interceptor auto-clears localStorage and dispatches logout() on any 401, immediately updating the Redux auth state and route guards.

10.2 Frontend Route Guards

Guard	Behavior
ProtectedRoute	Renders nested routes only if a Redux user is present; otherwise redirects to /login
AdminRoute	Redirects to /login if unauthenticated, to /dashboard if authenticated but not an admin

11. Controllers

Ten controllers implement all business logic. Every handler is wrapped in `asyncHandler` so thrown errors are forwarded to the centralized `errorHandler` instead of needing per-route `try/catch`.

Controller	Exports	Notes
<code>authController</code>	<code>register</code> , <code>login</code> , <code>getMe</code>	Registration bootstraps Portfolio + Watchlist for the new user
<code>userController</code>	<code>updateProfile</code> , <code>changePassword</code>	<code>changePassword</code> re-verifies the current password before hashing the new one
<code>stockController</code>	<code>getStocks</code> , <code>getStockById</code>	<code>getStockById</code> returns the most recent 3000 <code>MarketData</code> points, re-ordered chronologically
<code>tradingController</code>	<code>buyStock</code> , <code>sellStock</code>	Recomputes average price on BUY (weighted average), enforces wallet balance / holding quantity, tracks <code>totalInvestment</code> / <code>todayInvestment</code> ; <code>recalculatePortfolio()</code> syncs cached Portfolio totals
<code>portfolioController</code>	<code>getPortfolio</code>	Recomputes valuation, P/L (today & overall), allocation and net worth live on every request rather than trusting cached fields
<code>watchlistController</code>	<code>getWatchlist</code> , <code>addToWatchlist</code> , <code>removeFromWatchlist</code> , <code>updateWatchlistItem</code>	<code>updateWatchlistItem</code> derives <code>alertDirection</code> from the target price vs. current price and re-arms the alert on every edit
<code>goalController</code>	<code>getGoals</code> , <code>createGoal</code> , <code>updateGoal</code> , <code>deleteGoal</code>	<code>computeProgress()</code> derives live progress per goal type (wallet balance, portfolio value, profit, monthly investment total, or manual amount)
<code>transactionController</code>	<code>getTransactions</code>	Supports filtering by type and date range
<code>leaderboardController</code>	<code>getLeaderboard</code>	Ranks users by ROI computed fresh from holdings value + wallet balance vs. a fixed ₹1,00,000 baseline
<code>notificationController</code>	<code>getNotifications</code> , <code>markAllRead</code> , <code>clearNotifications</code>	Notifications are wiped on every server restart (<code>server.js</code>) — only alerts fired during the current run persist
<code>adminController</code>	<code>getDashboardStats</code> , <code>getUsers</code> , <code>updateUser</code> , <code>deleteUser</code> , <code>createStock</code> , <code>updateStock</code> , <code>deleteStock</code> , <code>getAllTransactions</code> , <code>getLogs</code>	Every mutating admin action is recorded via <code>logAction()</code> into <code>AdminLog</code>

12. Routes

Eleven Express routers are mounted in `app.js`. Each router pairs endpoints with the `protect` / `admin` middleware as required, delegating to the matching controller.

Router File	Mount Path	Guard
<code>authRoutes.js</code>	<code>/api/auth</code>	Mixed (<code>register/login</code> public, <code>/me</code> protected)

Router File	Mount Path	Guard
userRoutes.js	/api/users	protect
stockRoutes.js	/api/stocks	Public
tradingRoutes.js	/api/trading	protect
portfolioRoutes.js	/api/portfolio	protect
watchlistRoutes.js	/api/watchlist	protect
goalRoutes.js	/api/goals	protect
transactionRoutes.js	/api/transactions	protect
notificationRoutes.js	/api/notifications	protect
leaderboardRoutes.js	/api/leaderboard	protect
adminRoutes.js	/api/admin	protect + admin (router.use)

13. Models

Mongoose defines 11 schemas, all using { timestamps: true } (or createdAt-only where noted) for auditability. See Section 7.1 for full field-level detail; below is the module-facing summary.

Model File	Collection Purpose
User.js	Accounts, credentials, role, virtual wallet balance
Stock.js	Tradable instrument catalog with live price + company info
Portfolio.js	Per-user aggregate valuation and investment tracking
PortfolioHolding.js	Per-stock position within a portfolio (quantity, avg. price)
Watchlist.js / WatchlistItem.js	Per-user watchlist and its tracked stocks + alerts
Transaction.js	Immutable buy/sell ledger
Goal.js	User-defined financial targets
Notification.js	In-app price-alert notifications
MarketData.js	OHLCV history per stock for charting
AdminLog.js	Admin action audit trail

14. Implementation — Frontend

The client is a single-page React app. Redux Toolkit (6 slices: auth, stocks, portfolio, watchlist, goals, ui) manages global state; Axios (services/api.js) centralizes API calls and token handling; Framer Motion drives page transitions; React Three Fiber renders the landing page's 3D scene.

14.1 Landing Page

3D animated hero (HeroScene / PortfolioGlobe components) built with React Three Fiber, introducing the platform to unauthenticated visitors.



Figure 14.1 — Landing Page

14.2 Authentication

Split-panel auth layout (AuthLayout) for Login and Register; on success the JWT and user profile are persisted and Redux auth state is populated.

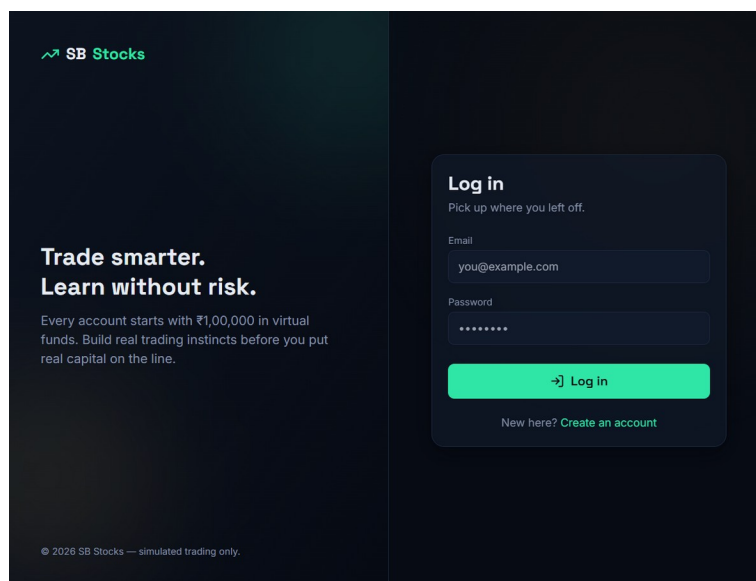
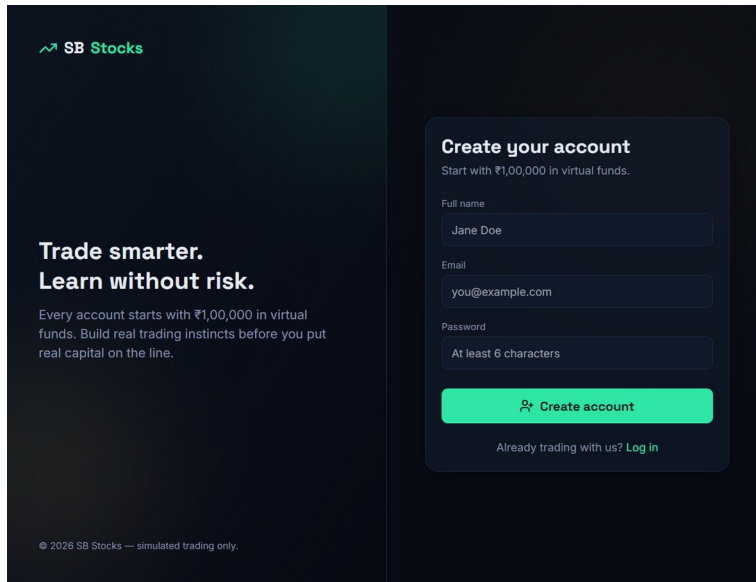


Figure 14.2 — Login Page



SB Stocks

**Trade smarter.
Learn without risk.**

Every account starts with ₹1,00,000 in virtual funds. Build real trading instincts before you put real capital on the line.

© 2020 SB Stocks — simulated trading only.

Create your account
Start with ₹1,00,000 in virtual funds.

Full name
Jane Doe

Email
you@example.com

Password
At least 6 characters

Create account

Already trading with us? [Log in](#)

Figure 14.3 — Register Page

14.3 Dashboard

Post-login landing view aggregating wallet balance, portfolio snapshot, quick trading achievements, and market highlights.

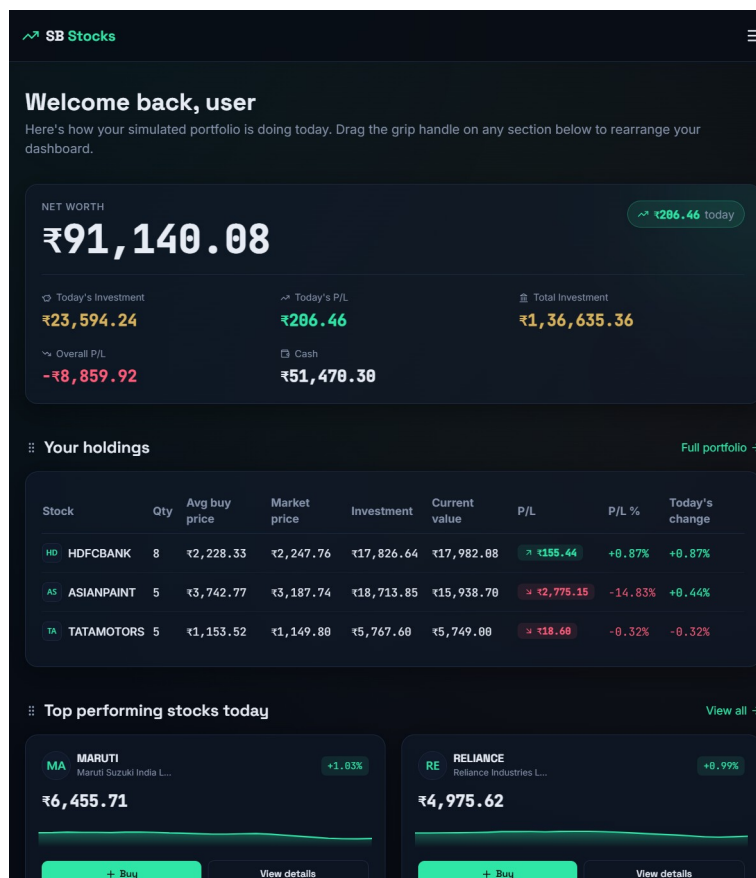


Figure 14.4 — Dashboard

14.4 Stocks & Comparison

Stocks page lists/searches/sorts the catalog (gainers, losers, volume); Compare lets users place stocks side-by-side across price and fundamental metrics.

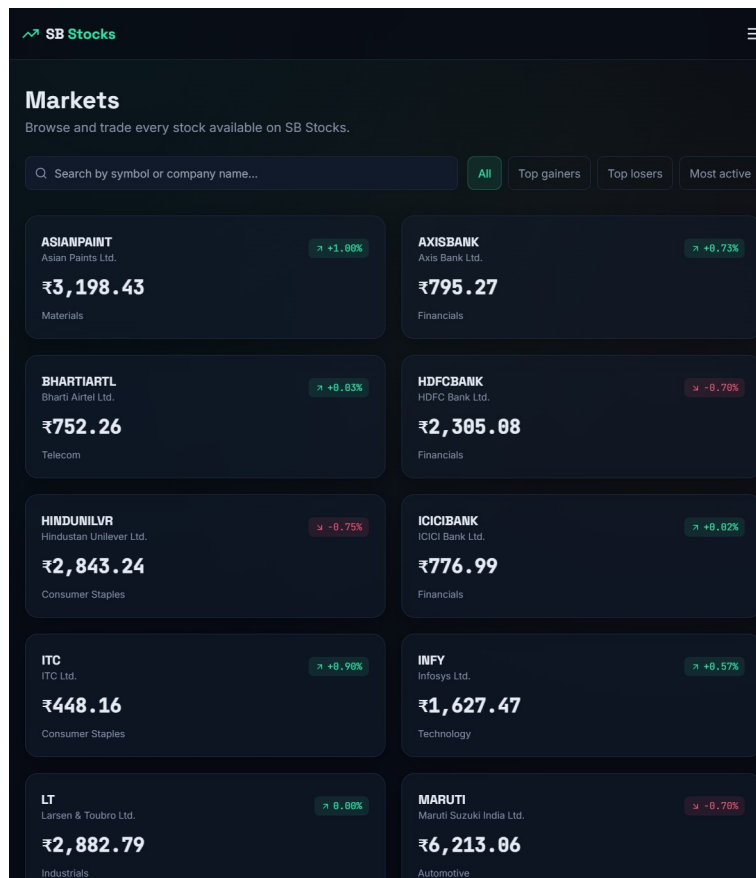


Figure 14.5 — Stocks Listing

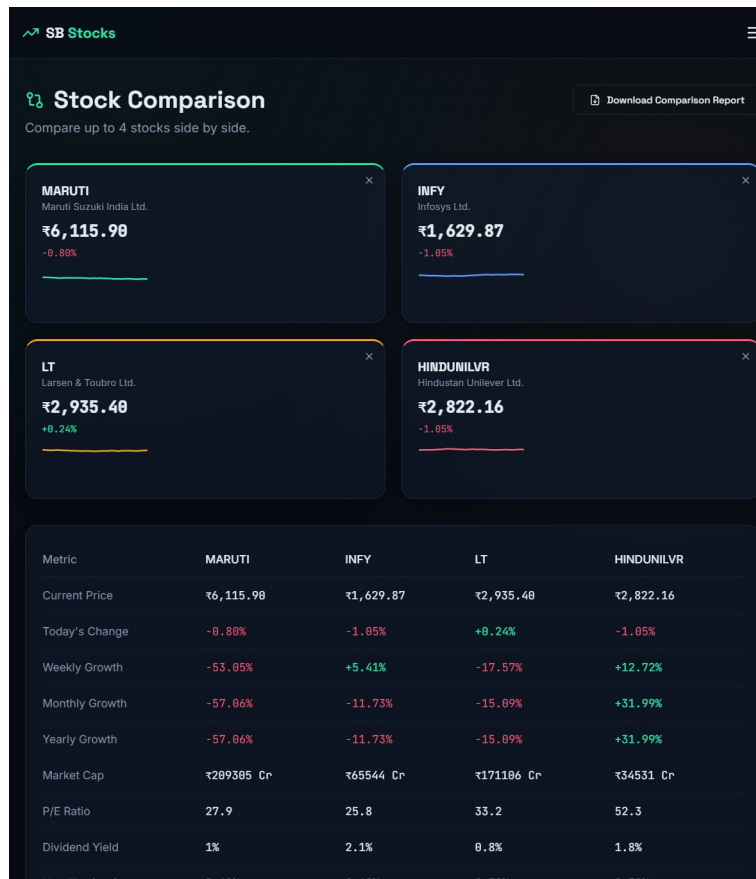


Figure 14.6 — Stock Comparison

14.5 Portfolio

Live holdings table, allocation chart, and performance chart, sourced directly from GET /api/portfolio on each load.

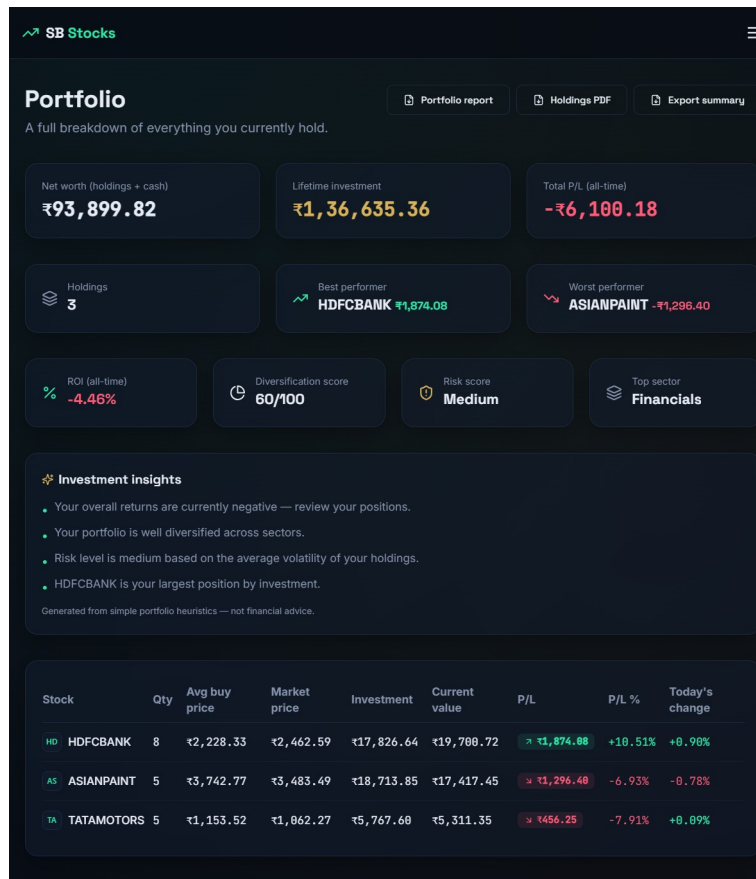


Figure 14.7 — Portfolio

14.6 Watchlist

Pin favorite stocks and configure above/below price-target alerts that surface as Notifications when triggered by the price-simulation tick.

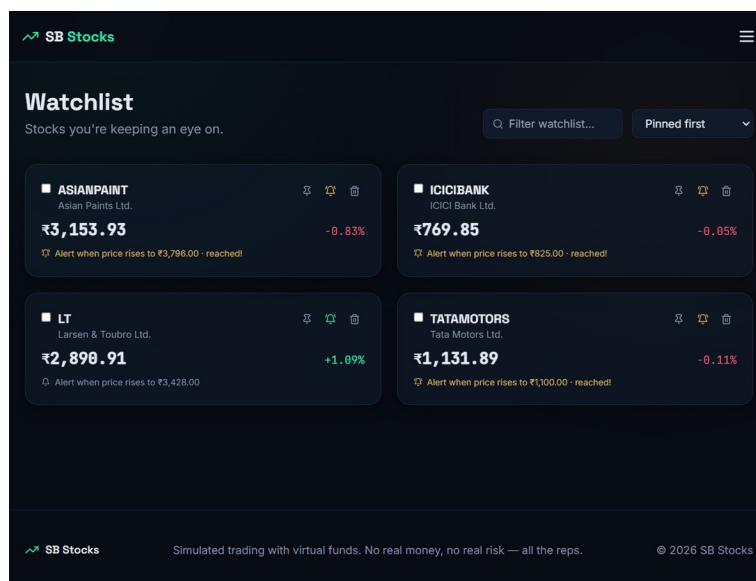


Figure 14.8 — Watchlist

14.7 Goals

Users define SAVINGS, PORTFOLIO_VALUE, PROFIT, MONTHLY_INVESTMENT, or CUSTOM goals; progress bars are computed live against real account data.

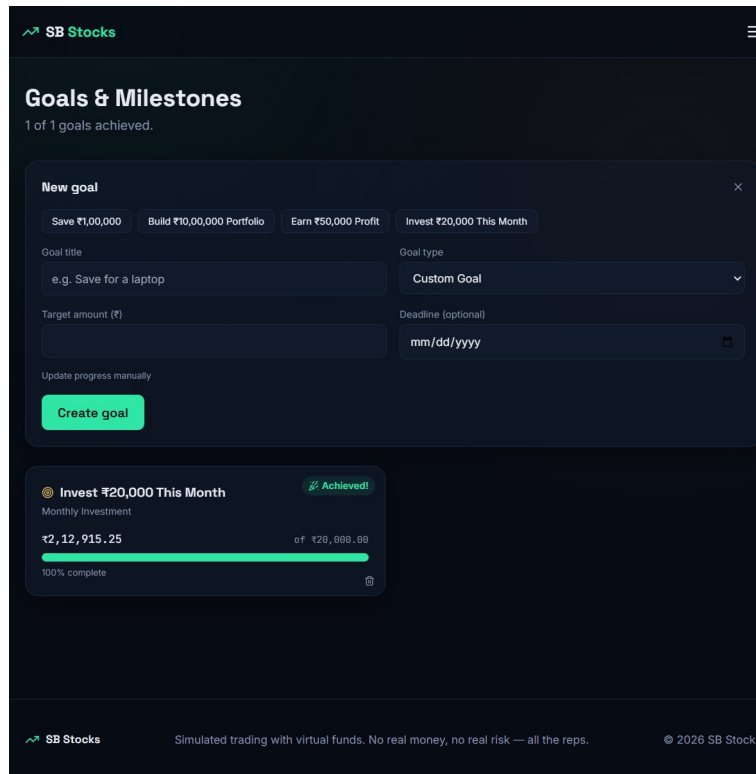


Figure 14.9 — Goals & Milestones

14.8 Leaderboard

Ranks all users by live ROI (net worth vs. the fixed ₹1,00,000 baseline), recalculated on every request.

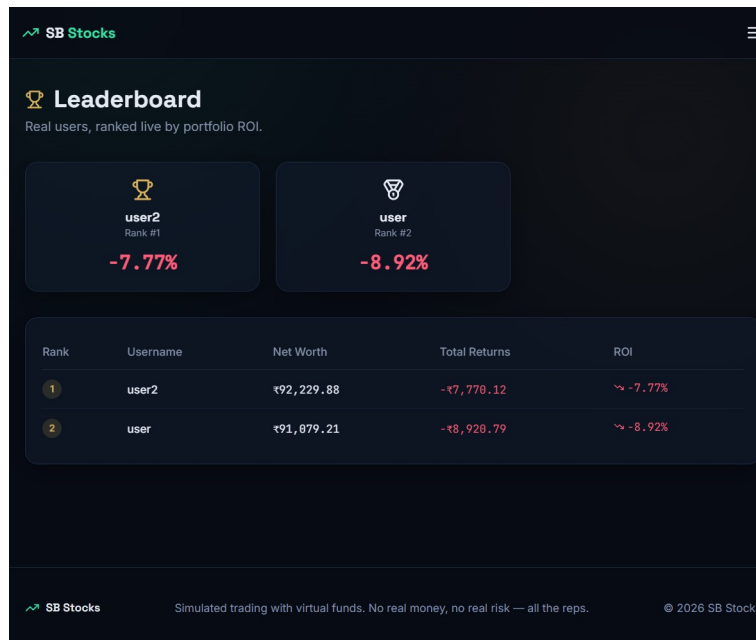


Figure 14.10 — Leaderboard

14.9 Transactions

Filterable buy/sell ledger (by type and date range), exportable via jsPDF (utils/pdfExport.js).

Type	Stock	Qty	Price	Total	Date
BUY	HDFCBANK	8	₹2,228.33	₹17,826.64	15 Jul 2026
BUY	TATAMOTORS	5	₹1,153.52	₹5,767.60	15 Jul 2026
SELL	AXISBANK	100	₹720.59	₹72,059.00	14 Jul 2026
BUY	AXISBANK	17	₹760.63	₹12,930.71	14 Jul 2026
BUY	AXISBANK	17	₹760.63	₹12,930.71	14 Jul 2026
BUY	AXISBANK	17	₹760.63	₹12,930.71	14 Jul 2026
BUY	AXISBANK	17	₹760.63	₹12,930.71	14 Jul 2026
BUY	AXISBANK	17	₹760.63	₹12,930.71	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026
BUY	AXISBANK	1	₹760.63	₹760.63	14 Jul 2026

Figure 14.11 — Transactions

14.10 Profile & Settings

Profile: update name/profile image and change password. Settings: application preferences (utils/preferences.js).

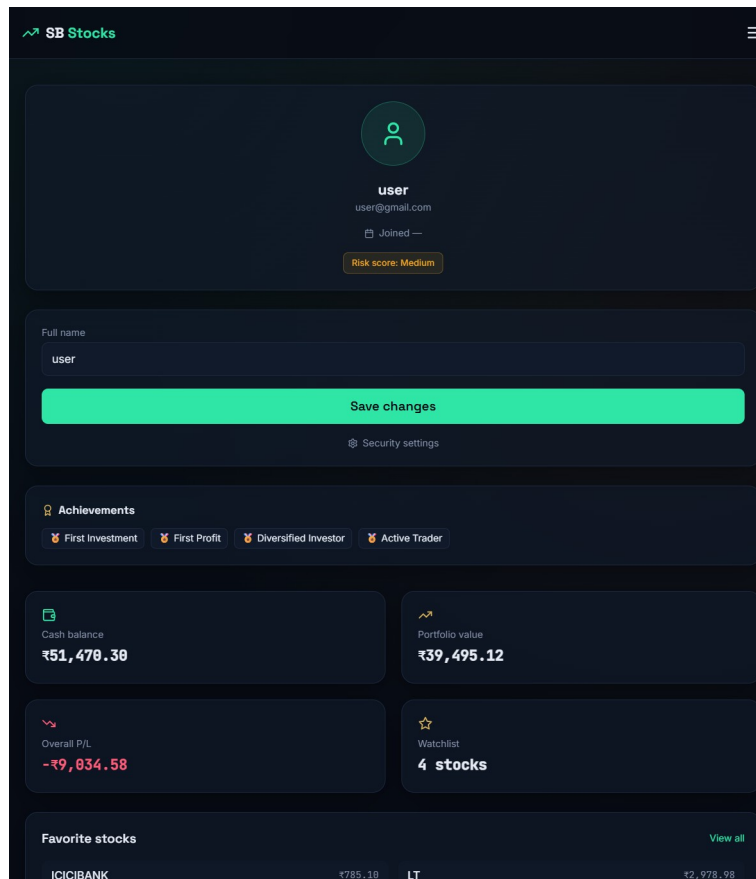


Figure 14.12 — Profile

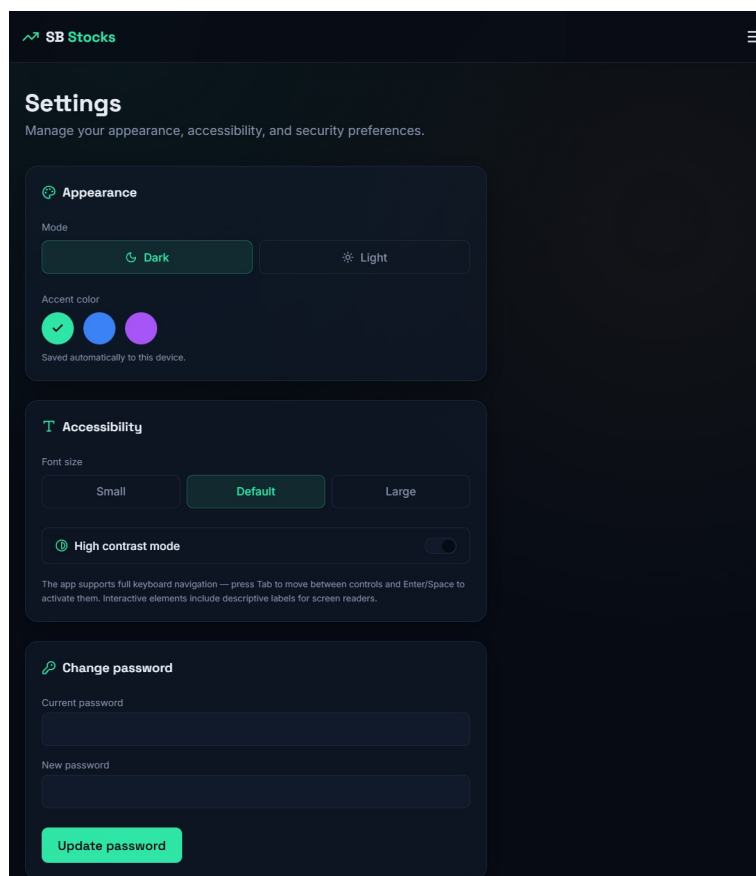


Figure 14.13 — Settings

15. Testing & Validation

15.1 Request Validation

- express-validator is included as a server dependency for structured request-body validation.
- Controllers additionally perform explicit guard checks — e.g. register requires name/email/password; buyStock/sellStock require a positive quantity and a resolvable stockId; createGoal requires title and targetAmount.
- Business-rule validation is enforced in controllers: insufficient wallet balance blocks a BUY; insufficient holding quantity blocks a SELL; duplicate email blocks registration; duplicate watchlist entries are rejected.

15.2 Error Handling

- asyncHandler forwards all async controller errors to a single errorHandler middleware, which returns a consistent { success: false, message } JSON shape (plus stack trace outside production).
- notFound middleware returns a structured 404 for any unmatched route.
- ErrorBoundary.jsx on the frontend catches render-time errors per route to avoid a blank-screen crash.

15.3 Manual / Functional Verification

- Auth flows: register → auto-created Portfolio/Watchlist → login → protected route access → 401 on expired/invalid token.
- Trading flows: buy at live price updates wallet, holding average price, and totalInvestment/todayInvestment; sell validates quantity and records costBasis for realized P/L.
- Watchlist alerts: setting a target price arms an alert; the 15-second price tick evaluates it via alertService and creates a Notification once satisfied.
- Admin flows: role/active toggling, stock CRUD, and platform analytics verified against seeded data (seed.js).

16. Security

Concern	Mitigation
Password storage	bcryptjs hashing (10 salt rounds) in a Mongoose pre-save hook; password field excluded from queries by default (select: false)
Session/auth tokens	Stateless JWTs signed with a server-side JWT_SECRET, fixed expiry (JWT_EXPIRES_IN)
Route protection	protect middleware verifies token + resolves the current user on every private request
Privilege separation	admin middleware restricts admin routes to role === "admin"; there is no public admin sign-up path
Account deactivation	isActive flag on User blocks login for deactivated accounts even with valid credentials
CORS	Configured via CLIENT_URL env var (app.js), restricting cross-origin access in production
Secrets management	MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, CLIENT_URL loaded from .env via dotenv, excluded from version control
Auditability	AdminLog records every admin mutation (actor, action, entity, details, timestamp)
Client-side token handling	Axios response interceptor auto-clears the token and Redux auth state on any 401, preventing use of a dead session

17. Conclusion

SB Stocks is a MERN-stack virtual trading simulator combining a real-time-feeling price engine, live-computed portfolio and leaderboard analytics, watchlist price alerts, goal tracking, and a role-based admin console — all backed by JWT authentication and a clean MVC-style Express API. The modular controller/route/model structure and clearly separated Redux-driven frontend make the platform straightforward to extend, including swapping the simulated price engine for a real market-data provider without changing the Stock schema shape.